



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +



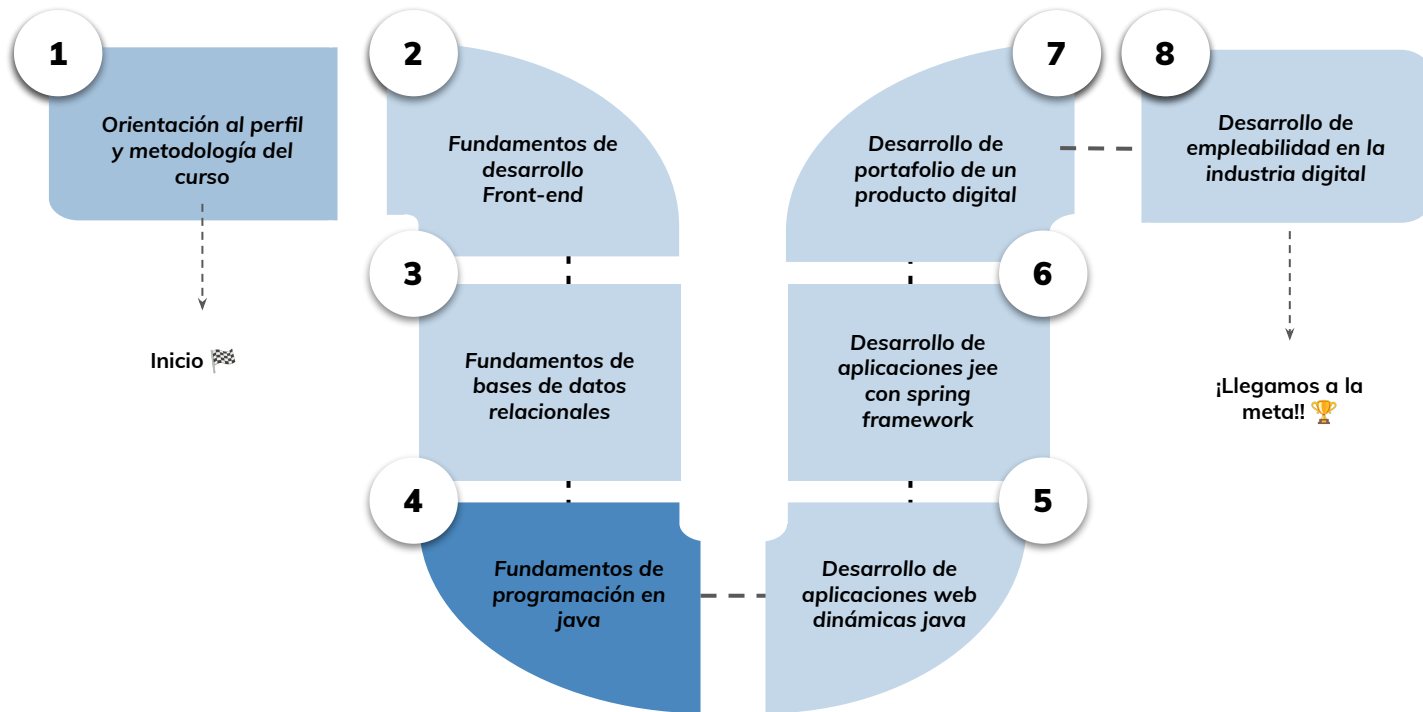
Pruebas unitarias en java-

› Parte 2

Aprendizaje Esperado: Implementar una suite de pruebas unitarias en lenguaje JAVA utilizando JUnit para asegurar el buen funcionamiento de una pieza de software.

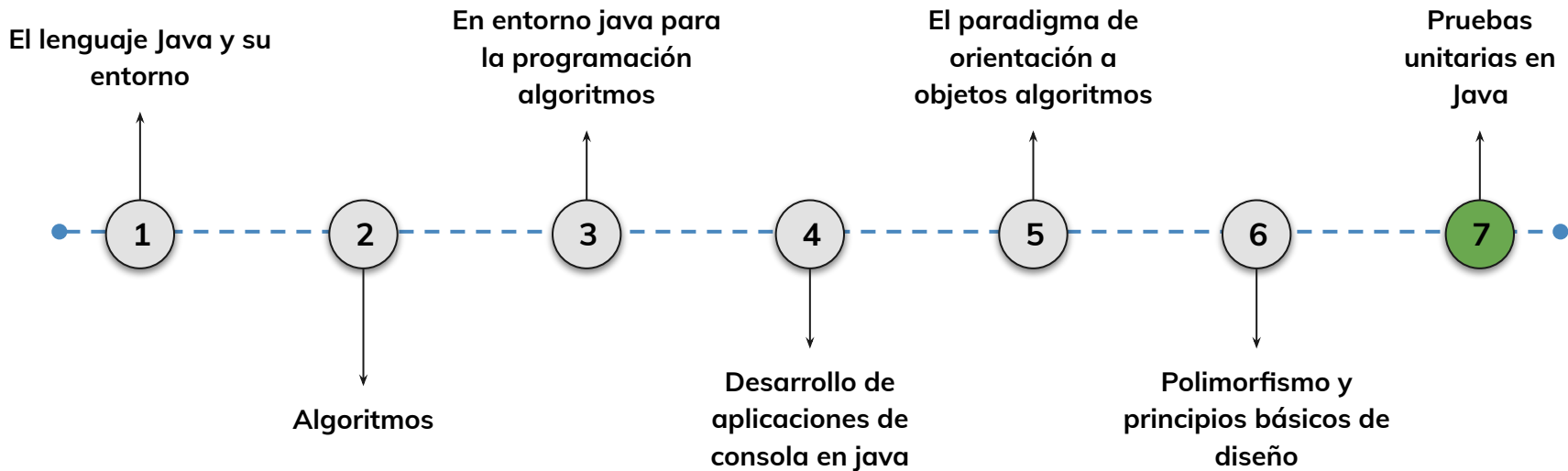
Hoja de ruta

¿Cuáles skills conforman el programa? **Desarrollo de Aplicaciones Full Stack Java Trainee**



Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

7.

Pruebas unitarias avanzadas con JUnit

Aprenderás a aplicar prácticas avanzadas en pruebas unitarias con JUnit, incluyendo fixtures, anotaciones, objetos simulados (mocks), el uso de Mockito y la construcción de suites de pruebas.

Anotaciones, Fixtures y Mocks

Uso de `@BeforeEach`, `@AfterAll`, `@Mock`, `@DisplayName`, entre otros.

Introducción a Mockito y uso de dobles de prueba (stub, spy, mock).

Suites de prueba con JUnit

¿Qué es una Test Suite?

Implementación con `@SuiteClasses` y ejecución agrupada de tests



Objetivos de aprendizaje

¿Qué aprenderás?



- Utilizar anotaciones de ciclo de vida en pruebas con JUnit.
- Aplicar técnicas de mocking con Mockito.
- Reconocer los tipos de dobles de prueba: dummy, fake, stub, spy, mock.
- Implementar mocks de forma manual y con anotaciones.
- Crear y ejecutar una suite de pruebas en Java

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Qué son los test unitarios, su importancia y características
- Ventajas y desventajas de implementar pruebas unitarias
- Introducción al framework JUnit y su ciclo de vida
- Creación de clases de prueba en Java usando Eclipse

Pruebas unitarias en java

➤ Utilización de Fixtures en las unidades de prueba

< > Utilización de Fixtures en las unidades de prueba

¿Qué son los Fixtures?:

Son objetos o estructuras de datos que se utilizan para **establecer un estado** inicial predefinido **antes** de ejecutar las pruebas y **restaurar el estado** original **después** de que se completen las pruebas.

Los fixtures son esenciales para garantizar que las pruebas se ejecuten de manera **aislada y consistente**.

En JUnit, los fixtures se definen utilizando **anotaciones** y métodos especiales que se ejecutan antes y después de cada prueba o de la suite de pruebas.



< > @Anotaciones

En JUnit5 encontraremos diversas anotaciones, las cuales empiezan con “@” y nos facilitan la escritura de los Test. Veamos algunas de ellas:

- **@Test**: Esta anotación se utiliza para marcar un método como una prueba unitaria. Los métodos anotados con @Test deben ser públicos y no devolver ningún valor.
- **@BeforeEach** : Esta anotación se utiliza para marcar un método que se ejecuta antes de cada prueba. Puedes usarlo para realizar configuraciones o inicializaciones necesarias antes de cada prueba.
- **@AfterEach** : Esta anotación se utiliza para marcar un método que se ejecuta después de cada prueba. Puedes usarlo para limpiar recursos o realizar acciones posteriores a cada prueba.

```
class EjemploTest {  
  
    // private MiClase miObjeto;  
  
    @BeforeEach  
    public void configuracionInicial() {  
        // Metodo que se correra antes de cada @Test  
        // miObjeto = new MiClase();  
    }  
  
    @AfterEach  
    public void limpiezaFinal() {  
        // Accion final que se ejecutara al final de cada @Test  
        // miObjeto = null; Liberar el objeto  
    }  
  
    @Test  
    void test1() {  
        int expected = 1;  
        int actual = 1;  
        assertEquals(expected, actual);  
    }  
  
    @Test  
    void test2() {  
        int unexpected = 2;  
        int actual = 1;  
        assertNotEquals(unexpected, actual);  
    }  
}
```

< > @Anotaciones

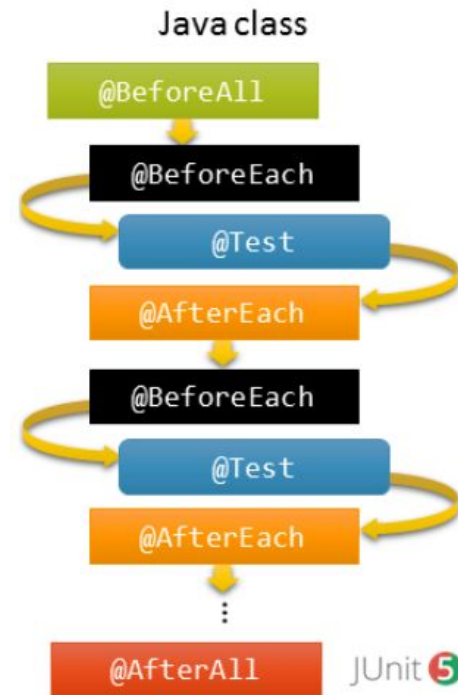
- **@BeforeAll** : Esta anotación se utiliza para marcar un método que se ejecuta una sola vez antes de todas las pruebas en una clase. El método anotado debe ser estático.
- **@AfterAll** : Esta anotación se utiliza para marcar un método que se ejecuta una sola vez después de todas las pruebas en una clase. El método anotado debe ser estático.

```
class EjemploTest {  
  
    // private MiClase miObjeto;  
  
    @BeforeAll  
    public void configuracionInicial() {  
        // Metodo que se corra una unica vez antes de que se corran los @Test  
        // miObjeto = new MiClase();  
    }  
  
    @AfterAll  
    public void limpiezaFinal() {  
        // Accion final que se corra cuando se corra el ultimo @Test  
        // miObjeto = null; liberar el objeto  
    }  
  
    @Test  
    void test1() {  
        int expected = 1;  
        int actual = 1;  
        assertEquals(expected, actual);  
    }  
  
    @Test  
    void test2() {  
        int unexpected = 2;  
        int actual = 1;  
        assertEquals(unexpected, actual);  
    }  
}
```

< > @Anotaciones

- **@DisplayName** : Se utiliza para proporcionar un nombre descriptivo para una prueba o una clase de prueba.
- **@Disabled** : Se utiliza para deshabilitar una prueba o una clase de prueba temporalmente sin eliminarla por completo.
- **@ParameterizedTest** : Permite ejecutar una misma prueba con diferentes conjuntos de parámetros.
- **@RepeatedTest** : Se utiliza para marcar un método como una prueba repetida y puedes especificar el número de repeticiones.
- **@Timeout** : Se utiliza para establecer un límite de tiempo para una prueba.

En la imagen podremos ver el orden de ejecución de los test:



LIVE CODING

Ejemplo en vivo

¡Vamos a colocar las anotaciones!

Es hora de anotar los métodos de nuestra clase Test

1. *Crear los métodos (sin implementar) y escribir las anotaciones necesarias de las clases Test que creamos en la clase anterior.*

Tiempo: 30 minutos

➤ Utilización de objetos simulados (mocks)

< > Utilización de objetos simulados (mocks)

¿Qué es el Mocking?:

Es una técnica utilizada en las pruebas unitarias para **simular el comportamiento de objetos reales** cuando la unidad que se está probando tiene dependencias externas. **Las simulaciones, o mocks, se utilizan en lugar de los objetos reales.**

El **objetivo de los mocks es aislar y concentrarse en el código que se está probando** y no en el comportamiento o estado de las dependencias externas.

Dentro de Spring Boot existe **Mockito** que nos ayuda a simular pruebas teniendo en cuenta, por ejemplo, qué tipos de datos enviamos y qué tipos de datos esperamos como respuesta.

En síntesis, es un tipo concreto de doble de test.



< > Dobles de prueba

Estos dobles son utilizados para hacer que las pruebas sean más fáciles de escribir, más rápidas o se posea un mayor control al simular elementos que no son el propósito principal de la prueba específica o que son costosos, en tiempo o dinero, de duplicar como un servicio externo o una base de datos.

- **Dummy**: se considera como argumento, pero nunca se usa realmente. Normalmente, los objetos dummy se usan solo para rellenar listas de parámetros.
- **Fake**: tiene una implementación que realmente funciona pero, por lo general, toma algún atajo o cortocircuito que le hace inapropiado para producción (como una base de datos en memoria, por ejemplo).



< > Dobles de prueba

Stubs

Son la forma más simple de los tres dobles de prueba. **Se parecen al código real que están reemplazando, pero siempre devuelven la misma respuesta estática**, independientemente de la entrada. Generalmente, es la mejor opción de doble cuando no le importa cuál es el comportamiento de la entidad duplicada. Solo desea que se ejecute sin causar una excepción.

Como los dobles del Museo de Cera de Madame Tussauds, los stubs **sirven a un único y sencillo propósito**.



< > Dobles de prueba

Spies

Es casi lo opuesto a los stubs. Permiten que la entidad duplicada conserve su comportamiento original al tiempo que proporciona información sobre cómo interactuó con el código bajo prueba. El spy puede decirle a la prueba qué parámetros se le dieron, cuántas veces se llamó y cuál fue el valor de retorno, si lo hubo.

Los spies también se pueden usar para simular o hacer doblaje de una pequeña parte de la entidad duplicada mientras se deja el resto del comportamiento de esa entidad en su lugar.

En el cine se utiliza la animatrónica para simular los movimientos de un actor en determinadas escenas, que podrían ser complejas o peligrosas.



< > Dobles de prueba

Mocks

Son los dobles de prueba más poderosos. Permiten un control completo sobre la entidad duplicada y proporcionan la misma información que proporciona un spy sobre cómo se ha interactuado con la entidad. Los mocks **se configuran antes de que se ejecute el código** bajo prueba para que se comporte como nos gustaría.

Hay todo tipo de situaciones para las que puedes configurar tus mocks. Algunas de estas situaciones incluyen:

- Devolver un valor esperado si se dan los parámetros correctos.
- Devolver un valor en la primera llamada y un valor diferente en la segunda llamada.
- Lanza una excepción.

Los Mocks son los verdaderos “dobles de acción”.

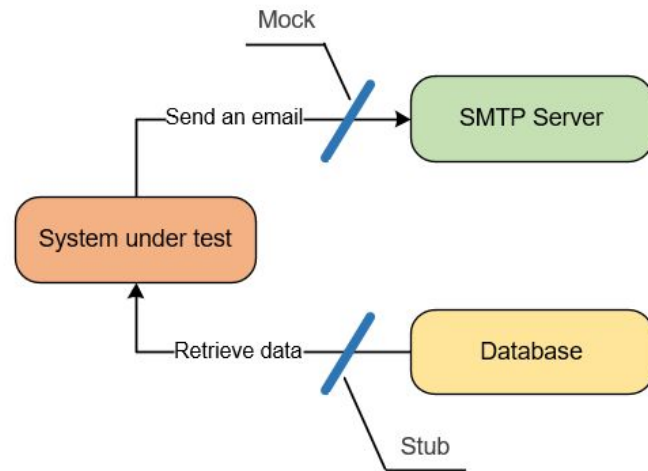


< > Utilización de objetos simulados (mocks)

Diferencia entre Stub y Mock

Aunque parezcan similares, podría decirse que el stub es como un mock con menor potencia, un subconjunto de su funcionalidad. Mientras que en el mock podemos definir expectativas con todo lujo de detalles, el stub tan solo devuelve respuestas preprogramadas a posibles llamadas.

Un **mock** valida comportamiento en la colaboración, mientras que el **stub** simplemente simula respuestas a consultas.



LIVE DEMO

Ejemplo en vivo

Creando mocks:

En este caso vamos a evaluar algunos métodos creados en clases anteriores.

1. Definir qué tipo de objetos simulados deberíamos crear (o elegimos crear) según los métodos/clases a testear

Tiempo: 40 minutos

➤ Utilizando Mockito

< > Mockito

Dentro de **Spring Boot** existe **Mockito** que nos ayuda a simular pruebas teniendo en cuenta, por ejemplo, qué tipos de datos enviamos y qué tipos de datos esperamos como respuesta.

Para implementar mockito en un proyecto:

En primer lugar, debe agregarse la dependencia.

Luego existen diferentes formas de habilitar el uso de anotaciones con las pruebas de Mockito:

- La primera opción que tenemos es anotar la prueba JUnit con un **MockitoJUnitRunner**
- Alternativamente, podemos habilitar las anotaciones de Mockito mediante programación invocando **MockitoAnnotations.openMocks()**

```
1 | @RunWith(MockitoJUnitRunner.class)
2 | public class MockitoTest {
3 |     ...
4 | }
```

```
1 | @Before
2 | public void init() {
3 |     MockitoAnnotations.openMocks(this);
4 | }
```



< > Mockito

Para crear un objeto simulado (mock) en un método:

Podemos usar el método **.mock()** de la clase **Mockito** para crear un objeto simulado de una clase o interfaz determinada. Esta es la forma más sencilla de simular un objeto.

```
1  @Test
2  public void test() {
3      List<String> mockList = Mockito.mock(List.class);
4      Mockito.when(mockList.size()).thenReturn(5);
5      assertTrue(mockList.size() == 5);
6  }
```



< > Mockito

También podemos simular un objeto usando la anotación @Mock.

Es útil cuando queremos usar el objeto simulado en varios lugares porque evitamos llamar al método mock() varias veces. El código se vuelve más legible y podemos especificar un nombre de objeto simulado que será útil en caso de errores.

```
1  @Mock
2  List<String> mockList;
3
4  @Test
5  public void test() {
6      when(mockList.get(0)).thenReturn("Hola Mundo");
7      assertEquals("Hola Mundo", mockList.get(0));
8  }
```



< > Mockito

when().thenReturn()

Los objetos simulados (mocks) pueden devolver diferentes valores según los argumentos pasados a un método. La cadena de métodos **when(...).thenReturn(...)** se utiliza para **especificar un valor de retorno para una llamada de método con parámetros predefinidos**.

```
1  @Test
2  void testeandoConWhenThenReturn() {
3
4      UsuarioDto esperado = UsuarioDto(0, "Juan");
5
6      // aqui simularemos que queremos que el método devuelva
7      Mockito.when(usuarioRepositorio.obtenerUsuario()).thenReturn(UsuarioDto(0, "Juan"));
8
9      UsuarioDto resultado = servicio.obtenerUsuario(0);
10     Assertions.assertEquals(esperado, resultado);
11 }
```



<> Mockito

Ejemplo de implementación

```
class CalculadoraTest {  
    @Mock  
    private Matematica matematica;  
  
    public CalculadoraTest() {  
        MockitoAnnotations.openMocks(this);  
    }  
  
    @Test  
    public void sumarRestar() {  
  
        // Configuración del mock  
        when(matematica.sumar(0,0)).thenReturn(100);  
        when(matematica.restar(0,0)).thenReturn(4);  
  
        // Creación del objeto bajo prueba  
        Calculadora calculadora = new Calculadora(matematica);  
  
        // Ejecución del método bajo prueba  
        int actual = calculadora.sumarRestar("Sumar", 50, 50);  
        assertEquals(10, actual);  
  
        actual = calculadora.sumarRestar("Restar", 12, 8);  
        assertEquals(0, actual);  
    }  
}
```

LIVE CODING

Ejemplo en vivo

Creando mocks:

En este caso vamos a implementar algunos mocks para manipular objetos simulados en el ejercicio que venimos trabajando.

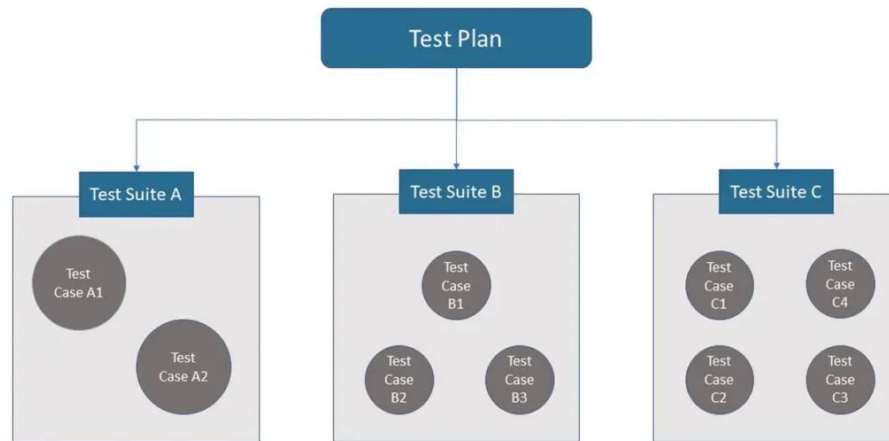
1. *Escribir las anotaciones necesarias de las clases Test: CuentaMockitoTest, TransaccionMockitoTest y MonedaMockitoTest*

Tiempo: 30 minutos

› Suites de pruebas (Test Suite)

< > Suites de pruebas

Las Test Suites en JUnit **son colecciones de casos de prueba que se agrupan para ejecutarse como una sola entidad**. Una suite de prueba permite organizar y ejecutar múltiples casos de prueba en secuencia o paralelamente, lo que facilita la ejecución y el análisis de pruebas en conjunto. En resumen, una suite de prueba **es una clase que actúa como contenedor para casos de prueba** JUnit.



< > Suites de pruebas

Características principales:

- **Agrupación de Casos de Prueba** : Las suites de prueba permiten agrupar casos de prueba relacionados o relevantes en una sola entidad.
- **Orden de Ejecución** : Puedes definir el orden de ejecución de los casos de prueba en una suite, lo que es útil cuando las pruebas dependen del estado resultante de pruebas anteriores.
- **Reusabilidad** : Las suites de prueba se pueden reutilizar en diferentes contextos y proyectos, lo que ahorra tiempo y esfuerzo en la configuración de las pruebas.
- **Separación de Pruebas** : Puedes crear múltiples suites de prueba para diferentes conjuntos de pruebas o escenarios de pruebas, lo que facilita la gestión de pruebas complejas.



< > Suites de pruebas

¿Cómo se plantean las Test Suites?

La forma en que se plantean las Test Suites **puede variar según la herramienta de prueba utilizada** . Veamos una guía general sobre cómo plantear una Test Suite:

- **Identificar casos de prueba relacionados** : Agrupa los casos de prueba que están relacionados y se centran en probar una funcionalidad o un componente específico.
- **Crear una clase de suite** que sirva como punto de entrada para la ejecución de la suite. Esta clase debe estar anotada adecuadamente según la herramienta de prueba que estés utilizando (por ejemplo `@ExtendWith` en JUnit 5).



< > Suites de pruebas

- **Especificar los casos de prueba incluidos** : En la clase de suite, especifica los casos de prueba individuales que deseas incluir en la suite. Puedes hacer esto utilizando anotaciones o métodos específicos proporcionados por la herramienta de prueba.
- **Configurar y ejecutar la suite** : Configurar cualquier configuración adicional necesaria para la suite y ejecútala. Esto puede implicar la configuración de parámetros de ejecución, establecimiento de datos iniciales, etc.

Una Test Suite es una colección de casos de prueba relacionados que se ejecutan juntos para probar una funcionalidad o un componente del software. Las suites de prueba son útiles para organizar, reutilizar y ejecutar eficientemente casos de prueba, y proporcionan informes agregados para una visión general de los resultados de las pruebas.



< > Suites de pruebas

Creación de Suites de Prueba en JUnit:

1. Crear una nueva clase que actúe como la suite de prueba. Esta clase no contiene lógica de prueba, pero se utiliza para agrupar casos de prueba relacionados.
2. Usar la anotación **@RunWith(Suite.class)** en la clase de la suite para indicar que es una suite de prueba.
3. Definir una lista de clases de casos de prueba a ejecutar dentro de la suite usando la anotación **@SuiteClasses({})** . Enumera las clases de casos de prueba separadas por comas.
4. Ejecutar la suite de prueba, y JUnit ejecutará todos los casos de prueba enumerados en la suite.



LIVE CODING

Ejemplo en vivo

Test Suites:

Vamos a crear una suite de prueba que contenga los casos de prueba codificados en las clases anteriores.

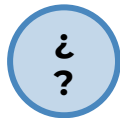
1. *Crear la suite y escribir las anotaciones necesarias para agrupar los casos de prueba que realizamos en las clases anteriores.*

Tiempo: 30 minutos

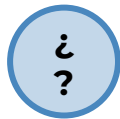
#Momentode Preguntas...



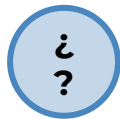
¿Qué diferencia clave hay entre un mock y un stub?



¿Qué tipo de doble usarías para simular una API externa costosa?



¿Qué función cumple @BeforeEach?



¿Cómo se relaciona una Test Suite con un pipeline de CI/CD?



Momento:

Time-out!



5 -10 min.





Ejercicio N° 1

Suite de pruebas para BilleteraVirtual

Suite de pruebas para BilleteraVirtual

Contexto: 🙌

Tu proyecto de billetera virtual ya cuenta con pruebas unitarias individuales. Ahora, debes organizar esas pruebas para que puedan ejecutarse todas juntas de manera automatizada.

Consigna: 📝

Agrupar las clases CuentaTest, TransaccionTest y MonedaTest en una única suite de prueba utilizando JUnit. Implementa el uso de mocks y fixtures cuando sea necesario. Asegúrate de configurar correctamente las anotaciones y ejecutar la suite desde Eclipse.

Tiempo 🕒: 35 minutos

Paso a paso: ⚙️

1. Crea la clase SuiteTest en un nuevo archivo Java.
2. Usa `@RunWith(Suite.class)` y `@SuiteClasses`.
3. Incluye las tres clases de test existentes.
4. Crea un test con `@Mock` y simula un comportamiento con `when().thenReturn()`.
5. Ejecuta toda la suite desde Eclipse con cobertura.

¿Alguna consulta?



Resumen

¿Qué logramos en esta clase?

- ✓ Comprender el uso de *Fixtures* en pruebas unitarias.
- ✓ Usar anotaciones como `@BeforeEach`, `@AfterAll`, `@DisplayName`.
- ✓ Reconocer y aplicar distintos tipos de dobles de prueba.
- ✓ Implementar mocks con Mockito.
- ✓ Crear una suite de prueba agrupada.
- ✓ Ejecutar y analizar múltiples pruebas en conjunto.
- ✓ Preparar el entorno para integración continua.



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

